

Code-Layout, Readability and Reusability

Date	01 July 2026
Team ID	
Project Name	Smart Lender
Maximum Marks	5Marks

Code-Layout, Readability and Reusability:

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Used proper Python indentation throughout the notebook.
2	Proper File Structure	Files and folders are logically organized	Yes	Dataset, Jupyter Notebook, trained model, and project files are organized properly.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Used descriptive variable names such as df, X, y, x_train, x_test, loan_status.
4	Function / Method Names	Functions are descriptively named	Yes	Used standard Scikit-learn and Pandas functions with clear naming conventions.
5	Code Comments	Inline and block comments explain logic	Yes	Added comments for preprocessing, encoding, balancing, scaling, and model training.
6	Modular Design	Code is split into reusable functions/modules	Partial	Code is organized into preprocessing, training, and prediction sections; further modularization can be done in the Flask application.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Removed unnecessary code and retained only required preprocessing and model-building steps.
8	Error Handling	Exceptions and errors are handled gracefully	Partial	Basic error checking was performed during preprocessing; additional exception handling will be included in the web application.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	Data Preprocessing Module	Python (Pandas, NumPy, Scikit-learn)	Data cleaning, encoding, scaling, feature engineering	High
2	SMOTE Balancing Module	Python (Imbalanced-learn)	Handling class imbalance in training data	High
3	Model Training Module	Python (Scikit-learn)	Training ML models (Logistic Regression, Decision Tree, Random Forest)	High
4	Model Evaluation Module	Python (Scikit-learn, Matplotlib, Seaborn)	Evaluating model performance (Accuracy, Confusion Matrix, Classification Report)	High
5	Prediction Module	Python (Pickle, NumPy)	Making predictions on new loan applicant data	High
6	Saved Model (model.pkl)	Python (Pickle)	Used in Flask web application for loan prediction	High
7	Scaler Object (scaler.pkl)	Python (Pickle)	Used in Flask app for scaling user input before prediction	High
8	Flask Application (app.py)	Python (Flask)	Web interface for taking user input and showing results	Medium
9	HTML Templates (index.html, result.html)	HTML, CSS	Displaying input form and prediction result in web app	Medium
10	Utility Functions	Python	Helper functions for preprocessing user input in Flask app	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	The project follows a well-structured and modular approach. Code is organized into clear sections (data preprocessing, balancing, model training, evaluation, prediction). Proper file structure maintained for the ML model and Flask app.
Readability	5	The code is clean, well-indented, and easy to read. Meaningful variable and function names are used throughout the project. Comments are added to explain important steps.
Reusability	4	Reusable components/modules are created for preprocessing, model training, and prediction. The saved model and scaler can be reused in the Flask web application for predictions.
Documentation / Comments	5	Inline comments and section-wise explanations are provided. Each major step is documented to make the code understandable for future reference.
Overall Score	4.75 / 5	The Smart Lender – Loan Prediction project is well structured, readable, and efficient with good documentation and reusability. Minor improvements can make it even better.

